# Frontend Development

Node, TypeScript, Nunjucks & GOV.UK Design System

# What We'll Cover

- HTML, CSS and front-end JS basics
- HTML templating with Nunjucks
- GOV.UK Design System
- Setting up GOV.UK Frontend
- Wiring up your frontend: controllers, routers & views

# HTML

HyperText Markup Language – the **structure** of web pages

- Uses **tags** to define elements: `<tagname>content</tagname>`
- Every page has `<html>`, `<head>`, and `<body>` sections
- **Semantic tags** describe meaning: `<header>`, `<nav>`, `<main>`, `<footer>`
- Attributes add extra info: `<a href="/about">About</a>`

```
<form action="/submit" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" />
  <button type="submit">Submit</button>
</form>
```

🔗 MDN HTML Reference

# CSS

Cascading Style Sheets – the **appearance** of web pages

- Controls colours, fonts, spacing, and layout
- **Selectors** target elements: `element`, `.class`, `#id`
- **Box model**: content → padding → border → margin

```
.button {
  background-color: #1d70b8; /* GDS blue */
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
}
```

> With GOV.UK Design System, most styling is handled for you!

🔗 MDN CSS Reference

# JavaScript

The **behaviour** and interactivity of web pages

- Runs in the browser (client-side) or on servers via Node.js
- Manipulates HTML via the **DOM**
- Handles **events**: clicks, form submissions, key presses

```
const button = document.querySelector(".submit-btn");
button.addEventListener("click", (event) ⇒ {
  event.preventDefault();
  console.log("Button clicked!");
});
```

> In server-rendered apps you'll write less client-side JS – the server does more work.

**Reference all JavaScript from external `.js` files** – never use inline `<script>` tags in HTML. This keeps your code organized, testable, and maintainable.

# JavaScript

**Fine to use JS for immediate page responses:**

- Progressive enhancements that improve existing HTML
- Client-side interactions that don't block page rendering
- Form validation and user feedback
- Critical rule: The server must always validate the password too.
- Never trust client-side validation for security. An attacker can bypass or disable the JS.

**Avoid JS for critical functionality:**

- The page must render and be usable without JavaScript
- Critical data submission should always go through the server
- Initial page load should not depend on JS execution

🔗 MDN JavaScript Reference

# HTML Templating

Generating HTML dynamically from data

**The problem:** Static HTML can't display dynamic content.

**The solution:** Templating engines combine **templates** + **data** → **HTML**

```
"Hello, {{ name }}!" + { name: "Sarah" } = "Hello, Sarah!"
```

**Why use templates?**

- **Reusability** – write once, use in multiple places
- **Maintainability** – layout is separate from code

**Popular engines:** Nunjucks, Handlebars, EJS, Pug

# Nunjucks

A powerful templating engine for JavaScript

- Created by **Mozilla**, inspired by Python's Jinja2
- Works with **Node.js** and in browsers
- Used extensively in **GOV.UK** projects

```
npm install nunjucks @types/nunjucks
```

**Key features:**

- `{{ }}` for variables
- Template inheritance with `extends` / `block`
- Macros (reusable template functions)
- Filters to transform data

🔗 Nunjucks Documentation

# Nunjucks Syntax

Variables, conditionals & loops

```
<!-- Variables -->
<h1>Hello, {{ user.name }}!</h1>

<!-- Conditionals -->
{% if user.isAdmin %}
  <a href="/admin">Admin Panel</a>
{% endif %}

<!-- Loops -->
{% for item in items %}
  <li>{{ item.name }}</li>
{% endfor %}
```

# Nunjucks Syntax

Filters, includes & variables

```
<!-- Filters -->
{{ name | upper }}          <!-- "sarah" → "SARAH" -->
{{ price | round(2) }}      <!-- 19.999 → 20.00 -->

<!-- Include a partial -->
{% include "partials/header.njk" %}

<!-- Set a variable -->
{% set greeting = "Welcome back" %}
<h1>{{ greeting }}, {{ user.name }}!</h1>

<!-- Comments (not rendered) -->
{# This won't appear in the HTML #}
```

# GOV.UK Design System

A design system for UK government services

- **Consistent** – same look and feel across all services
- **Accessible** – meets WCAG 2.1 AA out of the box
- **Tested** – components tested with real users
- **Maintained** – actively updated by GDS

**What it provides:**

- Components: buttons, forms, tables, error messages
- Patterns: common user flows
- Styles: typography, colours, spacing
- Nunjucks macros ready to use

🔗 design-system.service.gov.uk

# Setting Up GOV.UK Frontend

Four steps to get started

### 1. Install:

```
npm install govuk-frontend
```

### 2. Serve assets via Express:

```
app.use("/assets", express.static(
  path.join(__dirname, "node_modules/govuk-frontend/dist/govuk/assets")
));
```

### 3. Configure Nunjucks to find GDS templates:

```
nunjucks.configure(["views", "node_modules/govuk-frontend/dist"], { ... });
```

### 4. Import macros in templates:

```
{% from "govuk/components/button/macro.njk" import govukButton %}
```

# Using GDS Components

Building a page with GDS macros

```
{% extends "layouts/base.njk" %}
{% from "govuk/components/input/macro.njk" import govukInput %}
{% from "govuk/components/button/macro.njk" import govukButton %}

{% block content %}
  <h1 class="govuk-heading-l">Enter your name</h1>

  <form action="/submit" method="POST">
    {{ govukInput({
      label: { text: "Full name" },
      id: "full-name",
      name: "fullName"
    }) }}

    {{ govukButton({ text: "Continue" }) }}
  </form>
{% endblock %}
```

Macros accept objects with options — check the GDS docs for all available settings.

# Base Templates

Template inheritance with `extends` and `block`

```
<!DOCTYPE html>
<html lang="en" class="govuk-template">
<head>
  <title>{% block title %}My Service{% endblock %}</title>
  <link rel="stylesheet" href="/assets/govuk-frontend.min.css" />
</head>
<body class="govuk-template__body">
  {% include "partials/header.njk" %}

  <main class="govuk-main-wrapper">
    {% block content %}{% endblock %}
  </main>

  {% include "partials/footer.njk" %}
</body>
</html>
```

Child templates extend the base and override blocks — no repeated boilerplate!

# UI Project Structure

Organising your frontend code

```
src/
├── routes/              # Express route handlers
│   └── home.ts
├── views/               # Nunjucks templates
│   ├── layouts/         # Base templates
│   │   └── base.njk
│   ├── partials/        # Reusable fragments
│   │   ├── header.njk
│   │   └── footer.njk
│   └── pages/           # Page templates
│       ├── home.njk
│       └── form.njk
├── public/              # Static assets (CSS, JS, images)
└── app.ts               # Express app setup
```

# Exercise: Wiring Up the Frontend

Your starter project has the backend logic – you set up the frontend

**What's already done for you:**

- Express configured in `app.ts`
- An in-memory `ExpenseService` (returns hardcoded data – no database/backend endpoints needed)
- An `ExpenseController` with methods for all CRUD operations

**What you need to do:**

1. Install and configure **Nunjucks** as your templating engine
2. Install **GOV.UK Frontend** and serve the static assets via Express
3. Create a **base layout** template with the GOV.UK HTML structure
4. Add **header** and **footer** partials
5. Create an **expense router** wiring URLs to the provided controller methods
6. Register the router in `app.ts`
7. Create **Nunjucks templates** to render the expense data

# Controller → Router → View

How the pieces fit together

```
// expenseController.ts — already provided in the starter
export class ExpenseController {
  constructor(private expenseService: ExpenseService) {}

  async getAll(req: Request, res: Response): Promise<void> {
    const expenses = await this.expenseService.getAll();
    res.render("pages/expenses.njk", { expenses });
  }
}
```

```
// expenseRouter.ts — you create this
const controller = new ExpenseController(new ExpenseService());
router.get("/expenses", controller.getAll.bind(controller));
```

```
<!-- pages/expenses.njk — you create this -->
{% for expense in expenses %}
  <p>{{ expense.description }} — £{{ expense.amount }}</p>
{% endfor %}
```

# Key Takeaways

- **HTML** structures content, **CSS** styles it, **JS** adds behaviour
- **Nunjucks** lets you generate HTML dynamically from data
- **GOV.UK Design System** gives you accessible, consistent components for free
- The **MVC pattern** keeps your frontend organised: controller fetches data, router maps URLs, view renders HTML

# Over to You

Fork the starter repo, wire up the controller and router, and render your first GOV.UK page!